

MILESTONE 3 ASSIGNMENT 1

Implementation of Selenium Python and Behave BDD Framework

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Department of Computer Science and Engineering (AI & ML)
Institute of Engineering & Management (IEM), Newtown Campus

1. PROBLEM STATEMENT

In software quality engineering, manually validating end-to-end user journeys across iterative builds is inefficient and prone to human error. Furthermore, technical automation scripts often lack clarity for non-technical domain experts. This assignment addresses these challenges by configuring an automated BDD testing solution using **Python**, **Selenium WebDriver**, and **Behave Framework** to automate end-to-end web browser scenarios while maintaining plain-text Gherkin specifications.

2. OBJECTIVE

The core objectives of this assignment are:

- Set up a structured Python Behave BDD framework inside the PyCharm IDE environment.
- Write clear, executable test scenarios using Gherkin syntax (Given-When-Then).
- Implement Python step definition glue code using Selenium WebDriver.
- Configure environment hooks (`environment.py`) for clean browser instantiation and teardown.
- Execute end-to-end automation test suites and verify assertions against dynamic web elements.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Tool / Concept	Description & Purpose
Python 3.x	Core programming language utilized for writing automation logic and step definitions.
Behave	BDD framework for Python that parses Gherkin scenario files and runs matching Python code.
Selenium WebDriver	Browser automation API used to locate DOM elements and simulate real user actions.
PyCharm IDE	Development environment used for managing project files, package dependencies, and test execution.
Gherkin Language	Business-readable DSL providing structured keywords: Feature, Scenario, Given, When, Then.
WebDriver Manager	Automates binary executable management (e.g., ChromeDriver) without requiring manual driver paths.

Project Structure Note: BDD isolates business feature specifications from underlying automation drivers, ensuring tests are easy to read and maintain.

4. SOURCE CODE & IMPLEMENTATION STEPS

Step 1: Gherkin Feature File (features/login.feature)

features/login.feature	Gherkin Spec
<pre>Feature: User Authentication System Scenario: Successful login with valid user credentials</pre>	

Step 2: BDD Environment Hooks (features/environment.py)

features/environment.py	Python Hook
<pre>from selenium import webdriver from selenium.webdriver.chrome.service import Service from</pre>	

Step 3: Step Definitions Glue Code (features/steps/login_steps.py)

features/steps/login_steps.py	Python Logic
<pre>from behave import given, when, then from selenium.webdriver.common.by import By LOGIN_U</pre>	

```
@when('I click on the login button') def step_impl(context): context.driver.find_element
```

5. EXECUTION OUTPUT

```
Feature: User Authentication System Scenario: Successful login with valid user credentia
```

Figure 1.1: Standard Console Output Lines for Executed Behave Feature

6. RESULT

Execution Summary: The test scenario was executed through the Behave runner. The automated Chrome browser opened, navigated to the web application, filled in the login credentials, submitted the form, and successfully validated the authentication banner message.

Observation: Structuring tests with Behave BDD improves readability and separates concerns cleanly. The browser setup and teardown handled inside `environment.py` ensured that browser sessions closed properly without leaving lingering processes.

Conclusion: The setup of the Python Selenium Behave BDD automation framework was completed successfully. All automated steps executed without errors and met all specified requirements.